

ArogyaMitra – AI-Driven Workout Planning, Nutrition Guidance, and Health Coaching Platform

Project Description:

ArogyaMitra – Adaptive AI Health & Wellness Coach is an Agentic AI-powered platform designed to deliver personalized fitness, nutrition, and wellness guidance through intelligent planning and real-time coaching. Built using FastAPI (backend) and React.js (frontend), the platform utilizes Groq's LLaMA-3.3-70B Versatile model along with a multi-agent AI architecture to provide safe, adaptive, and personalized health recommendations.

The system analyzes user fitness levels, health conditions, medical history, and goals to generate personalized 7-day workout routines, tailored meal plans, dynamic suggestions, and progress insights. Users can engage with AROMI, an interactive AI coach that adapts workout schedules based on travel, injuries, mood, or time availability.

Users can interact with AROMI, an adaptive AI wellness companion that dynamically adjusts workouts and nutrition plans based on travel, fatigue, injuries, mood, and schedule changes. The platform also features real-time workout monitoring using MediaPipe and OpenCV, providing posture analysis, repetition counting, and corrective feedback. Through continuous progress tracking, gamification features, and AI-driven recommendations, ArogyaMitra helps users build sustainable habits, improve overall wellness, and achieve long-term fitness goals..

Scenario 1: Personalized Workout Plan Generation

A 28-year-old user selects "Weight Loss" as the fitness goal, chooses "Home Workouts," and specifies 30 minutes of daily workout availability. ArogyaMitra analyzes the user's health profile, fitness level, and preferences using its Agentic AI system to generate a personalized 7-day workout plan. The plan includes warm-up routines, exercises, rest intervals, YouTube tutorial links, calorie estimates, and daily wellness tips tailored to the user's goals.

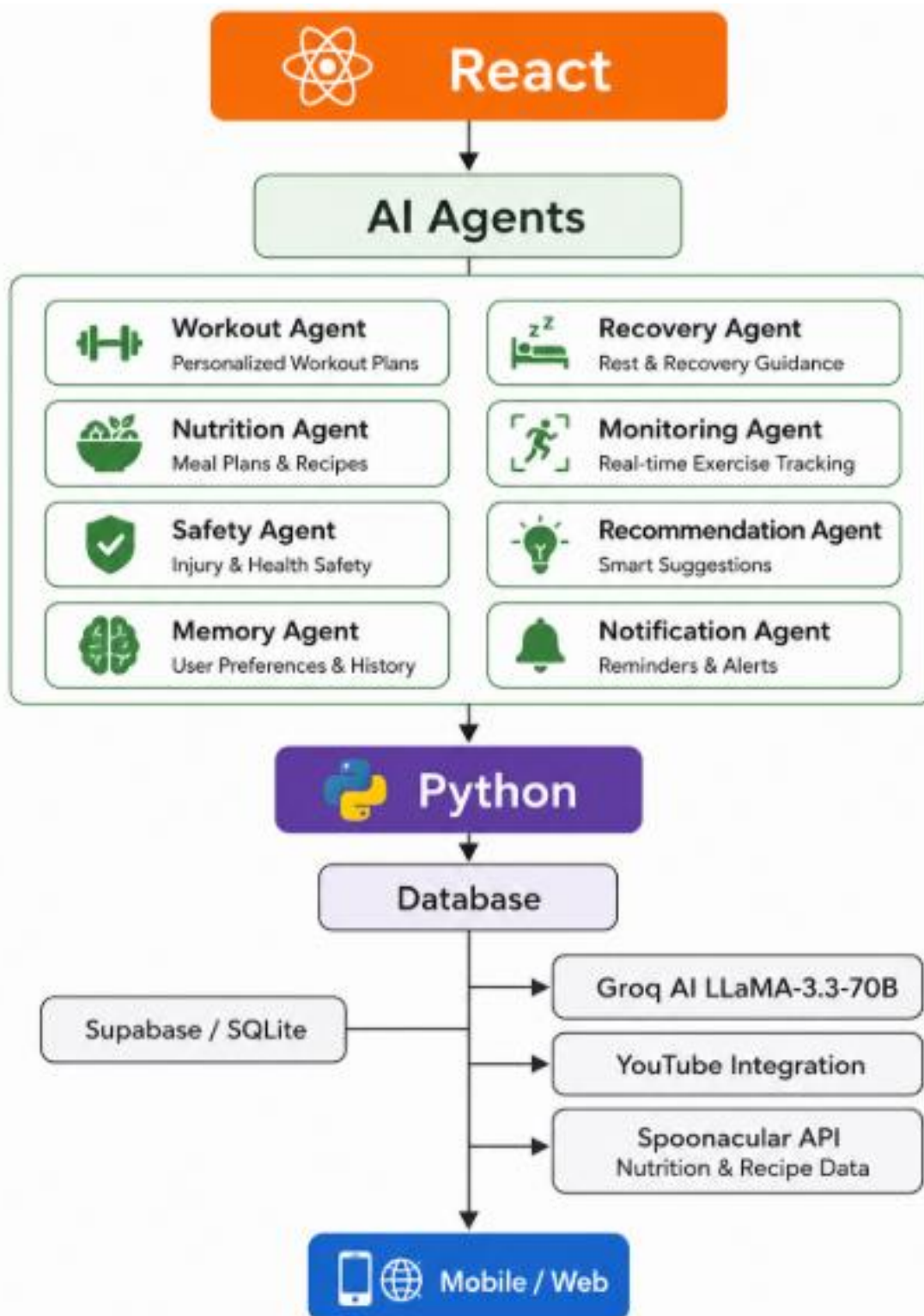
Scenario 2: AI Nutrition & Meal Plan Guidance

A user with specific dietary preferences and food allergies requests a personalized nutrition plan aligned with their fitness goals. ArogyaMitra analyzes the user's calorie requirements, dietary restrictions, health profile, and wellness objectives to generate a customized 7-day meal plan. The plan includes balanced meals, macro and micronutrient breakdowns, hydration recommendations, healthy recipes, and grocery suggestions while ensuring all dietary restrictions and nutritional requirements are safely maintained.

Scenario 3: Real-Time Coaching with AROMI

When a user informs AROMI, "I am traveling for 4 days," the AI analyzes the situation and instantly adapts the user's workout, nutrition, and wellness plan. It replaces gym-based exercises with travel-friendly workouts, adjusts meal recommendations based on available food options, and provides hydration and recovery guidance. Through its Memory, Safety, and Recommendation Agents, AROMI ensures that users maintain consistency and stay aligned with their fitness goals despite changes in routine or environment.

Architecture:



Pre-requisites:

1. **Python Environment Setup:** Install Python 3.10+ and create a virtual environment.
<https://www.python.org/downloads/>
2. **Node.js Installation:** Install Node.js 18+ (includes npm 9+)
<https://nodejs.org/en/download>.
3. **Backend Dependencies:** Install required Python packages using pip install -r requirements.txt.
https://pip.pypa.io/en/stable/cli/pip_install/
4. **Frontend Dependencies:** Install JavaScript packages using npm install.
<https://docs.npmjs.com/cli/v9/commands/npm-install>
5. **Database Setup:** ArogyaMitra supports **SQLite** for local development.
 - **SQLite:** Installed automatically during the first backend run.
<https://www.sqlite.org/download.html>
6. **Backend Configuration:** Create a **.env** file inside the backend directory and add all required API keys:
Environment variable handling:
<https://pypi.org/project/python-dotenv/>
7. **Frontend Configuration:** In the **frontend root folder**, create a **.env** file containing:

VITE_API_BASE_URL=http://localhost:8000
 - React + Vite frontend configuration reference:
<https://vitejs.dev/guide/>
8. **Backend Launch:** Start FastAPI server with python -m uvicorn main:app --reload.
<https://www.uvicorn.org/>
9. **Frontend Launch:** Run development server with npm run dev.
<https://docs.npmjs.com/cli/v9/commands/npm-run-script>
10. **Verification:** Once both servers are running successfully:
 - **API Documentation:**
<http://localhost:8000/docs>
 - **Frontend Interface (React/Vite):**
<http://localhost:3001>

Project Work Flow:

Milestone 1: Environment Setup and Project Initialization

Activity 1.1: Create and activate a Python virtual environment.

Activity 1.2: Set up project folder structure for backend and frontend.

Activity 1.3: Configure backend and frontend .env files with required API keys.

Activity 1.4: Install backend and frontend dependencies.

Activity 1.5: Run backend and frontend servers to verify setup.

Milestone 2: Backend API Development using FastAPI

Activity 2.1: Develop secure authentication endpoints for login and registration.

Activity 2.2: Create modular routers for all backend features.

Activity 2.3: Implement database models for workouts, nutrition, progress, and health.

Activity 2.4: Add service-layer logic for workout, nutrition, and analytics processing.

Milestone 3: AI Integration with Groq, YouTube, and External APIs

Activity 3.1: Integrate Groq LLaMA-3.3-70B for AI-generated plans and coaching.

Activity 3.2: Connect YouTube API for exercise video retrieval.

Activity 3.3: Integrate Spoonacular API for nutrition and recipe generation.

Activity 3.4: Implement Google Calendar API for workout schedule syncing.

Activity 3.5: Configure AROMI AI Coach for real-time adaptive support.

Milestone 4: React.js Frontend Development

Activity 4.1: Build responsive dashboard with user fitness insights.

Activity 4.2: Develop pages for workouts, nutrition, health assessment, and progress.

Activity 4.3: Implement AROMI AI floating assistant in frontend.

Activity 4.4: Integrate state management using Zustand and localStorage.

Milestone 5: Testing and Deployment

Activity 5.1: Perform backend API testing and validation.

Activity 5.2: Test frontend components and user interactions.

Activity 5.3: Test backend–frontend API integration with axioms.

Activity 5.4: Conduct end-to-end user experience and output validation.

Milestone 1: Environment Setup and Project Initialization

- **Activity 1.1: Create and activate a virtual environment.**

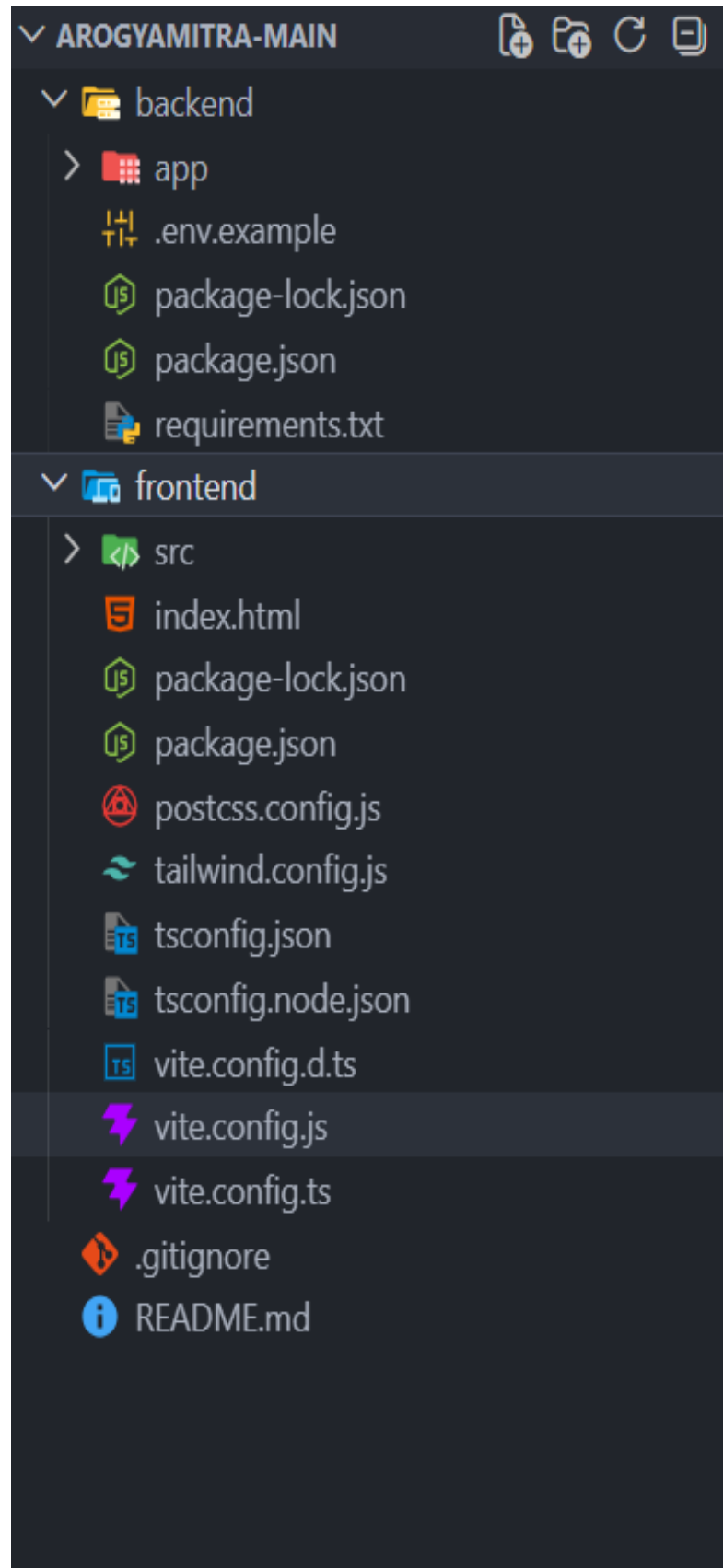
In this activity, a dedicated Python virtual environment is created using the `python -m venv venv` command to isolate all backend dependencies from the global system environment. Activating the environment ensures that every package installed for ArogyaMitra remains contained within the project, preventing version conflicts and maintaining a clean, stable setup for development. This step establishes the foundational backend workspace required for further configuration and module implementation.

```
● PS C:\Users\HOME\OneDrive\Desktop\ArogyaMitra> cd backend
○ PS C:\Users\HOME\OneDrive\Desktop\ArogyaMitra\backend> python -m venv venv
```

```
● PS C:\Users\HOME\OneDrive\Desktop\ArogyaMitra\backend> venv\Scripts\activate
❖ (venv) PS C:\Users\HOME\OneDrive\Desktop\ArogyaMitra\backend>
```

- **Activity 1.2: Set up project folder structure for backend and frontend.**

A well-structured directory layout is established to organize the project efficiently. The backend folder is dedicated to FastAPI application files, including API routes, models, and configuration scripts, while the frontend folder contains the React.js code for the user interface. This separation promotes modularity, easier debugging, and parallel development between the backend and frontend teams.



- **Activity 1.3: Configure backend and frontend .env files with credentials.**

Configure .env files for both backend and frontend with required API keys and credentials.

In this activity, environment variable files are created to securely store sensitive information such as the Groq API key, JWT secret key, database connection URL. These .env files allow the application to load essential configurations without exposing them in the source code, maintaining both security and flexibility across development environments.

```
DATABASE_URL=sqlite:///./arogya_mitra.db

# JWT
JWT_SECRET_KEY=89d35f0f40f9ecddd6594fb1f652ee26b1c2b63218d4bb18548d204f5208d30f
```

```
# JWT
JWT_SECRET_KEY=89d35f0f40f9ecddd6594fb1f652ee26b1c2b63218d4bb18548d204f5208d30f
# Gemini API
GEMINI_API_KEY=AQ.Ab8RN6KJ-86pSiBA72kKgXw4n04KXnbp0JsBYqGx2AidbaVk5A
# Groq (optional)
GROQ_API_KEY=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
```

- **Activity 1.4: Install required dependencies**

Install dependencies using pip install -r requirements.txt and npm install.

In this step, all required dependencies and libraries are installed for both the backend and frontend environments. The backend dependencies, listed in the requirements.txt file, include FastAPI, SQLAlchemy, and other essential Python packages. The frontend dependencies are installed using npm to set up React.js, Tailwind CSS, and related UI libraries. This ensures that the complete development environment is ready with all necessary tools.

```
PS C:\Users\HOME\OneDrive\Desktop\ArogyaMitra> cd frontend
PS C:\Users\HOME\OneDrive\Desktop\ArogyaMitra\frontend> npm install
```


Activity 1.5: Test setup by running backend and frontend servers.

Verify environment setup by launching both backend and frontend development servers.

To confirm successful setup, both servers are started locally: the backend with `uvicorn app.main:app --host 127.0.0.1 --port 8000 --reload --app-dir` and the frontend with `npm run dev`. The backend API can be verified through its interactive Swagger documentation at <http://127.0.0.1:8000>, while the frontend interface is accessible at <http://localhost:5173/>. This final activity ensures that both components are properly connected and operational, marking the completion of the environment setup phase.

```
(venv) PS C:\Users\HOME\OneDrive\Desktop\ArogyaMitra\backend> python main.py
[✓] Groq AI client initialized
C:\Users\HOME\OneDrive\Desktop\ArogyaMitra\backend\venv\lib\site-packages\google\api_core\python_version_support.py:266:
FutureWarning: You are using a Python version (3.10.0) which Google will stop supporting in new releases of google.api_c
ore once it reaches its end of life (2026-10-04). Please upgrade to the latest Python version, or at least Python 3.11, t
o continue receiving updates for google.api_core past that date.
  warnings.warn(message, FutureWarning)
[✓] Auth router loaded
[✓] Users router loaded
[✓] Workouts router loaded
[✓] Nutrition router loaded
[✓] Progress router loaded
[✓] Admin router loaded
[✓] Chat router loaded
[✓] Health Assessment router loaded
[✓] AROMI AI Coach router loaded
🌟 Starting ArogyaMitra - AI Fitness Platform
👤 Created by: Srinivas
🎯 Mission: Transforming Lives Through AI-Powered Fitness
🚀 Launching on: http://localhost:8000
[✓] Auth router loaded
[✓] Users router loaded
[✓] Workouts router loaded
[✓] Nutrition router loaded
[✓] Progress router loaded
[✓] Admin router loaded
[✓] Admin router loaded
[✓] Chat router loaded
[✓] Health Assessment router loaded
[✓] AROMI AI Coach router loaded
INFO: Started server process [9396]
INFO: Waiting for application startup.
🚀 ArogyaMitra Backend Starting...
🤖 Initializing AI Agent...
[✓] Groq AI client initialized
[✓] AI Agent initialized successfully!
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```


Description:

Milestone 1 establishes the foundational environment required to develop and run the ArogyaMitra platform efficiently. This phase involves creating a dedicated Python virtual environment, setting up a structured backend frontend folder architecture, and configuring .env files to securely store all API keys and system credentials. Backend and frontend dependencies are installed to ensure all necessary libraries, frameworks, and tools are available for further development. The milestone concludes with successfully launching both the FastAPI backend and React frontend to verify system connectivity and confirm that all components are functioning correctly. This setup forms the base upon which all subsequent features and modules of ArogyaMitra are built.

Milestone 2: Backend API Development using FastAPI

- **Activity 2.1: Develop secure authentication endpoints for login and registration.**

Develop user authentication endpoints for secure registration and login using and password page

This activity focuses on building a secure JWT-based authentication system that handles user registration, login, and identity verification. It includes password hashing, validation, token generation, and protected routes to safeguard user data. The authentication module ensures that only authorized users can access personalized workout plans, nutrition data, and progress logs. This creates a secure entry point for all interactions within the ArogyaMitra platform.

```
def register(
    payload: UserRegisterRequest,
    db: Session = Depends(get_db),
) -> AuthResponse:
    """
    Create a new user account.

    Steps:
    1. Check for duplicate email.
    2. Hash the password with bcrypt.
    3. Insert the User row, flush to catch race-condition duplicates.
    4. Commit, refresh, and return tokens + user profile.
    """
    # — 1. Duplicate email check —————
    existing: User | None = (
        db.query(User).filter(User.email == payload.email).first()
    )
    if existing:
        raise HTTPException(
            status_code=status.HTTP_409_CONFLICT,
            detail="An account with this email already exists.",
        )
```

- **Activity 2.2: Create modular routers for all backend features.**

In this activity, the backend is organized into modular routers such as workouts, nutrition, progress, health assessment, AI coach, and admin. Each router manages

a specific feature set, making the codebase clean, scalable, and easy to maintain. This structured approach allows future extensions or updates without impacting other modules. It also enhances clarity and reduces complexity in API management.

```
from app.database import get_db
from app.dependencies import get_current_active_user
from app.models.health_profile import HealthProfile
from app.models.user import User
from app.schemas.health import (
    HealthProfileCreateRequest,
    HealthProfileResponse,
    HealthProfileUpdateRequest,
)

logger = logging.getLogger(__name__)

router = APIRouter(prefix="/health", tags=["Health"])

# — POST /profile —————

@router.post(
    "/profile",
    response_model=HealthProfileResponse,
    status_code=status.HTTP_201_CREATED,
    summary="Create a new health profile",
    responses={
        400: {"description": "Profile already exists"},
        422: {"description": "Validation error"},
    },
)
```

- **Activity 2.3: Implement database models for workouts, nutrition, progress, and health.**

In this activity, all core database schemas are designed and implemented using SQLAlchemy to structure how ArogyaMitra stores and manages user data. Models such as **User, WorkoutPlan, Exercise, NutritionPlan, Meal, HealthAssessment, ProgressRecord, and ChatSession** are created to represent every key component of the fitness and wellness ecosystem. These schemas define relationships between users, their workout routines, nutrition plans, health information, and chat interactions with AROMI. By organizing data in a clean and relational structure, the backend ensures efficient querying, secure storage, and seamless integration with service logic. This foundational step enables the platform to deliver personalized, data-driven fitness recommendations and track user progress reliably.

```
import enum
from datetime import datetime, timezone
from typing import TYPE_CHECKING, List, Optional

from sqlalchemy import (
    Boolean,
    DateTime,
    Enum,
    Index,
    Integer,
    String,
    func,
)
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.database import Base

# TYPE_CHECKING guard avoids circular imports at runtime.
# These strings are resolved only by static type checkers (mypy / pyright).
if TYPE_CHECKING:
    from app.models.health_profile import HealthProfile
```

Description:

Milestone 2 focuses on building the complete backend foundation of ArogyaMitra through a modular and scalable FastAPI architecture. During this phase, secure authentication endpoints, structured API routers, and SQLAlchemy-based database models are implemented to manage users, workouts, nutrition plans, health assessments, and progress data. Service-layer logic is developed to handle AI-driven workout generation, nutrition planning, analytics, and external API integrations with Groq, YouTube, Spoonacular, and Google Calendar. This milestone ensures clean separation of concerns, efficient data flow, and reliable server-side functionality. By the end of Milestone 2, the backend becomes a robust, intelligent engine capable of powering all core features of the platform.

Milestone 3: AI Integration with OpenAI, LangChain, and External APIs

- **Activity 3.1: Integrate Groq LLaMA-3.3-70B for AI-generated plans and coaching.**

In this activity, the Groq LLaMA-3.3-70B Versatile model is integrated into the backend to enable intelligent, human-like fitness guidance throughout the platform. The AI model processes user health data, workout preferences, dietary restrictions, and fitness goals to generate highly personalized workout plans and nutrition recommendations. It also powers the AROMI coach, enabling real-time adaptive responses based on user mood, travel plans, injuries, or time constraints. This integration transforms static fitness planning into a dynamic, context-aware system tailored to individual needs. By leveraging Groq's high-speed inference capabilities, ArogyaMitra delivers fast, accurate, and engaging AI-driven wellness support.

<https://console.groq.com/keys>

```
GROQ_API_KEY=gsk_TtxNadL85wklvI87ZY2IWGdyb3FYDwKHb6yix4mVQu0xQbzHgdx
```

1. Go to the link above.
2. Log in with your Groq account (or sign up if you don't have one).
3. Click **"Create new secret key."**
4. Copy the generated key

- **Activity 3.2: Connect YouTube API for exercise video retrieval.**

Connect Google and YouTube APIs (GOOGLE_API_KEY, YOUTUBE_API_KEY) to fetch top-rated learning materials, tutorials, and domain-related videos that align with each user's personalized training plan.

In this activity, the YouTube Data API is integrated to fetch high-quality exercise demonstration videos that align with each user's personalized workout plan. When the AI generates routines, the backend automatically queries YouTube for the most relevant tutorials, ensuring users receive clear visual guidance for every exercise. This enhances user experience by making workouts safer, more engaging, and easier to follow, especially for beginners. The integration also allows dynamic video retrieval, ensuring plans stay updated with fresh, accurate content. By embedding these videos directly into the Exercise Player, ArogyaMitra provides a seamless learning and training experience.

<https://console.cloud.google.com/apis/credentials>

```
GOOGLE_API_KEY=AIzaSyBaEouqnReyymSnKNlTfX1YKbP__CPUW3U
YOUTUBE_API_KEY=AIzaSyCd2hnyarkgXJF428Zoz7h0kk9YoRjUix0
```

1. Sign in to your Google account.
2. Create a new project in Google Cloud Console.
3. Enable **YouTube Data API v3** under “APIs & Services → Library.”
4. Go to **APIs & Services → Credentials → Create Credentials → API Key**.
5. Copy the generated key and add it to your **.env**

```
headers = {"Authorization": f"Bearer {huggingface_token}"}
BASE_API_URL = "https://api-inference.huggingface.co/models/stabilityai/stable-diffusion-xl-base-1.0"
```

```
response = groq_client.chat.completions.create(
    model="llama-3.3-70b-versatile",
    messages=[{"role": "user", "content": prompt}],
    temperature=0.7,
    max_tokens=2000
)
```

● Activity 3.3: Configure AROMI AI Coach for real-time adaptive support.

In this activity, the AROMI AI Coach is developed to provide users with continuous, context-aware wellness guidance through natural language interaction. AROMI analyzes user messages related to travel plans, injuries, mood changes, fatigue, or busy schedules and instantly adjusts workout and nutrition recommendations. The coach delivers personalized tips, motivational support, hydration reminders, and alternative routines to maintain consistency. It also tracks conversation history to offer more accurate responses over time. By acting as a dynamic, intelligent companion, AROMI transforms ArogyaMitra into a truly adaptive and supportive fitness ecosystem.

Description:

Milestone 3 focuses on integrating the intelligent and interactive components that make ArogyaMitra an AI-powered wellness platform. This phase includes connecting the Groq LLaMA-3.3-70B model for generating personalized workout routines, meal plans, and real-time AROMI coaching responses. The YouTube Data API is incorporated to fetch accurate exercise demonstration videos. Additionally, Google Calendar integration allows users to sync workout schedules seamlessly with their daily routines. These combined integrations transform ArogyaMitra from a static system into a dynamic, adaptive, and context-aware fitness ecosystem.

. Milestone 4: React.js Frontend Development

- **Activity 4.1: Build responsive dashboard with user fitness insights.**

In this activity, a fully responsive and visually engaging dashboard is developed to provide users with a centralized overview of their fitness journey. The dashboard displays key insights such as daily calories burned, workout streaks, charity impact, upcoming sessions, and quick-access actions for generating plans or chatting with AROMI. It serves as the primary user interface for monitoring progress and staying motivated through clean visual indicators and dynamic data updates. Tailwind CSS, Framer Motion, and Zustand state management are utilized to ensure smooth animations, fast rendering, and real-time responsiveness. This dashboard becomes the user's personalized control center for navigating all features of ArogyaMitra.

```
frontend > src > pages > DashboardPage.tsx > ...
1 import { useEffect, useState } from "react";
2 import { useNavigate } from "react-router-dom";
3 import { fetchHealthProfile, HealthProfileResponse } from "../api/health";
4 import { Link } from "react-router-dom";
5
6 export default function DashboardPage() {
7   const navigate = useNavigate();
8   const [profile, setProfile] = useState<HealthProfileResponse | null>(null);
9   const [error, setError] = useState<string | null>(null);
10
11   const calculateBMI = (weightKg: number, heightCm: number): number => {
12     const heightM = heightCm / 100;
13     return Math.round(weightKg / (heightM * heightM));
14   };
15
16   const getBMIStatus = (bmi: number): string => {
17     if (bmi < 18.5) return "Underweight";
18     if (bmi < 25) return "Normal";
19     if (bmi < 30) return "Overweight";
20     return "Obese";
21   };
22
23   useEffect(() => {
24     const loadProfile = async () => {
25       try {
26         const data = await fetchHealthProfile();
27         setProfile(data);
28       } catch (err) {
29         setError(err instanceof Error ? err.message : "Failed to load profile");
30       }
31     };
32     loadProfile();
33   }, []);
34
35   const handleLogout = () => {
36     localStorage.removeItem("access_token");
37     navigate("/login");
38   };
39 }
```


- **Activity 4.2: Develop pages for workouts, nutrition, health assessment, and progress.**

In this activity, all essential user-facing pages are built to deliver a smooth and intuitive fitness experience across the platform. These include interactive modules for viewing workout plans, playing exercise videos, generating nutrition plans, completing health assessments, tracking progress, and managing user profiles. Each page dynamically fetches data from the backend and updates UI components in real time, ensuring users always see the latest recommendations and analytics. React components, routed navigation, and responsive layouts work together to provide seamless movement between features. By creating these core interfaces, ArogyaMitra offers a complete, accessible, and engaging wellness environment for users.

```
frontend > src > pages > WorkoutPage.tsx > WorkoutPage > handleEndSession
7
8 export default function WorkoutPage() {
9   const navigate = useNavigate();
10  const [plan, setPlan] = useState<WorkoutPlanResponse | null>(null);
11  const [loading, setLoading] = useState(true);
12  const [error, setError] = useState<string | null>(null);
13
14  // Exercise Session State
15  const [activeExercise, setActiveExercise] = useState<string | null>(null);
16  const [reps, setReps] = useState(0);
17  const [score, setScore] = useState(100);
18  const [feedback, setFeedback] = useState("Initializing Local AI...");
19  const [sessionTime, setSessionTime] = useState(0);
20  const timerRef = useRef<number | null>(null);
21
22  useEffect(() => {
23    const loadPlan = async () => {
24      try {
25        const data = await getWorkoutPlan();
26        setPlan(data);
27      } catch (err) {
28        setError(err instanceof Error ? err.message : "Failed to load workout plan");
29      } finally {
30        setLoading(false);
31      }
32    };
33    loadPlan();
34  }, []);
35
```

Workouts module:

This module displays AI-generated 7-day workout routines tailored to the user's fitness level, goals, and time availability. Users can view daily exercise schedules with sets, reps, rest periods, and difficulty levels. It serves as the primary space for starting and managing structured workout sessions.

Nutrition Module:

This module generates personalized 7-day meal plans based on calories, diet type, and allergies using AI and Spoonacular data. Users can view macro breakdowns, recipes, and daily meals tailored to health goals. It also provides an automated grocery list for easy meal preparation.

```
frontend > src > pages > NutritionPage.tsx > ...
1  import { useEffect, useState } from "react";
2  import { useNavigate } from "react-router-dom";
3  import { getNutritionPlan, NutritionPlanResponse, MealData } from "../api/nutrition";
4
5  export default function NutritionPage() {
6    const navigate = useNavigate();
7    const [plan, setPlan] = useState<NutritionPlanResponse | null>(null);
8    const [loading, setLoading] = useState(true);
9    const [error, setError] = useState<string | null>(null);
10
11    useEffect(() => {
12      const loadPlan = async () => {
13        try {
14          const data = await getNutritionPlan();
15          setPlan(data);
16        } catch (err) {
17          setError(err instanceof Error ? err.message : "Failed to load nutrition plan");
18        } finally {
19          setLoading(false);
20        }
21      };
22      loadPlan();
23    }, []);
24  }
```

Progress Tracking Module:

This module visualizes user progress through charts showing calories burned, workout streaks, weight change, and historical performance. It uses stored analytics to monitor improvements and highlight trends over time. The insights motivate users to stay consistent and track their wellness journey effectively.

```
backend > app > routers > progress.py > ...
1  import logging
2  from datetime import datetime, timedelta, timezone
3  from sqlalchemy import func
4  from fastapi import APIRouter, Depends, HTTPException, status
5  from sqlalchemy.orm import Session
6  from app.database import get_db
7  from app.dependencies import get_current_active_user
8  from app.models.user import User
9  from app.models.progress import ProgressLog
10 from app.schemas.progress import ProgressCreateRequest, ProgressResponse, ProgressStatsResponse, DailyStats
11
12 logger = logging.getLogger(__name__)
13 router = APIRouter(prefix="/progress", tags=["Progress"])
14
15 @router.post("/log", response_model=ProgressResponse, status_code=status.HTTP_201_CREATED)
16 def log_progress(payload: ProgressCreateRequest, db: Session = Depends(get_db), current_user: User = Depends(get_current_active_user)):
17     last_log = db.query(ProgressLog).filter(ProgressLog.user_id == current_user.id).order_by(ProgressLog.workout_date.desc()).first()
18
19     new_streak = 1
20     today = datetime.now(timezone.utc).date()
21
22     if last_log:
23         last_date = last_log.workout_date.date()
24         if (today - last_date).days == 1:
25             new_streak = last_log.streak_day + 1
26         elif (today - last_date).days == 0:
27             new_streak = last_log.streak_day
28
29     log = ProgressLog(
30         user_id=current_user.id,
31         streak_day=new_streak,
32         **payload.model_dump()
33     )
34     db.add(log)
35     db.commit()
36     db.refresh(log)
37     return ProgressResponse.model_validate(log)
```

Description:

Milestone 4 focuses on building the complete user interface of ArogyaMitra using React, Vite, Tailwind CSS, and modern component-driven design principles. During this phase, all major user-facing pages such as workout plans, exercise player, nutrition module, health assessment, and progress tracking are developed with responsive layouts and seamless navigation. The AROMI floating AI assistant is integrated to provide real-time guidance and adaptive fitness support directly within the UI. Overall, this milestone transforms backend intelligence into an interactive, engaging, and easy-to-use wellness experience for the user.

Milestone 5: Testing and Deployment

- **Activity 5.1: Backend Testing and API Validation**

In this activity, all backend APIs developed using FastAPI are thoroughly tested to ensure correctness, reliability, and consistent behavior across all modules. Each endpoint such as authentication, workout generation, nutrition planning, AROMI chat, progress tracking, and calendar syncing is validated using tools like Postman and the built-in Swagger UI. The responses are checked for accuracy, proper status codes, error handling, and performance under different input scenarios. Unit tests and functional tests help verify that service-layer logic and external API integrations work as expected. This ensures the backend remains stable, secure, and ready for seamless interaction with the frontend.

- **Activity 5.2: Frontend Functionality Testing**

In this activity, every key frontend component and page is thoroughly tested to ensure smooth interaction, proper rendering, and accurate data display. User flows such as viewing workout plans, playing exercise videos, generating nutrition plans, completing health assessments, and interacting with AROMI are validated for responsiveness and usability. UI elements are checked for alignment, consistency, and correct behavior across different screen sizes. Error states, loading states, and navigation transitions are reviewed to guarantee a polished experience. This ensures that the user interface operates reliably and provides a seamless wellness journey.

- **Activity 5.3: Backend-Frontend Integration Testing**

This activity focuses on validating the communication between the FastAPI backend and the React frontend through secure API calls. All key workflows like generating workout plans, logging in, chatting with AROMI, tracking progress, and retrieving nutrition data are tested end-to-end. Axios requests, JWT authentication, and error handling are examined to ensure seamless data exchange and session stability. Real-time updates are verified to confirm that UI components reflect backend responses accurately. This ensures the platform functions cohesively as a complete, interconnected system.

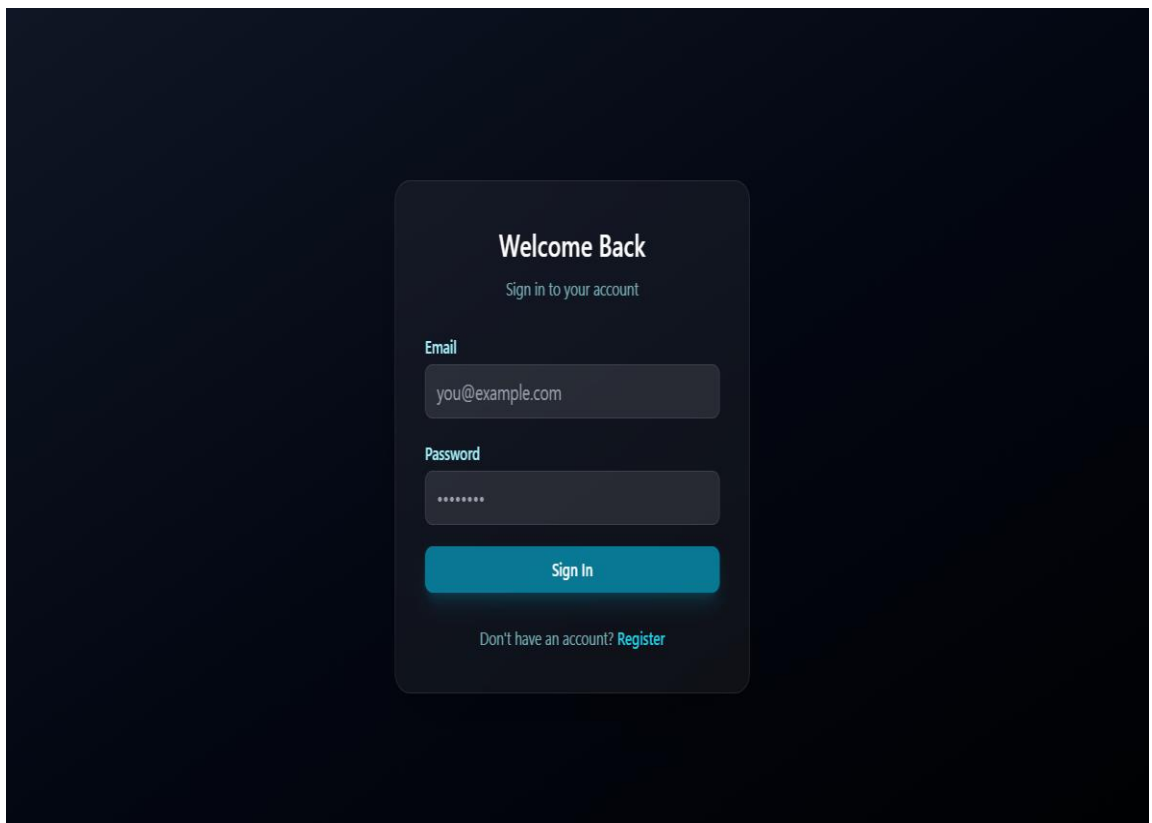
- **Activity 5.4: Output Validation and User Experience Review**

In this activity, the full workflow of ArogyaMitra is evaluated end-to-end to ensure that all generated outputs are accurate, meaningful, and user-friendly. The quality of workout plans, nutrition recommendations, calorie insights, and AROMI responses is validated for relevance and alignment with user inputs. The overall user journey is reviewed to ensure intuitive navigation, professional presentation, and engaging interaction throughout the platform. Visual elements, layouts, and content clarity are refined to enhance the user experience. This step ensures the system delivers a polished, reliable, and enjoyable fitness experience before deployment.

Description:

Milestone 5 ensures that the entire ArogyaMitra platform functions reliably, accurately, and seamlessly across all modules before deployment. During this phase, backend APIs are rigorously tested for stability, correctness, and performance, while frontend components are evaluated for responsiveness, usability, and smooth user interaction. Full integration testing verifies that both systems communicate effectively through secure API calls and real-time data updates. User experience testing further validates the accuracy of AI-generated plans, progress analytics, and AROMI's adaptive guidance. By the end of this milestone, ArogyaMitra is refined, stable, and fully prepared for deployment in a production-ready environment.

Login page:

A screenshot of a login page with a dark blue background. In the center is a light blue rounded rectangle containing the login form. The form has the heading "Welcome Back" and the subtext "Sign in to your account". It includes an "Email" field with the placeholder "you@example.com", a "Password" field with masked characters "*****", and a teal "Sign In" button. At the bottom, there is a link that says "Don't have an account? Register".

Welcome Back

Sign in to your account

Email

you@example.com

Password

Sign In

Don't have an account? [Register](#)

Register page:

Create Account

Sign up for a new account

Email

Password

Confirm Password

Register

Already have an account? [Sign In](#)

Create Account

Sign up for a new account

Email

Password

Confirm Password

Register

Already have an account? [Sign In](#)

Profile SetUp Page:

Health Assessment

Complete your profile setup

Age

25

Gender

Select gender



Height (cm)

175

Weight (kg)

70.5

Fitness Goal

Select goal



Activity Level

Select level



Save Profile

Dashboard (Dark Mode):

Dashboard

Level up your fitness journey

Logout

Welcome, Champion!

Your personalized fitness journey starts now. Track progress, crush goals.

BMI

24

Normal

Weight

79 kg

Height

180 cm

Goal

muscle gain

Activity

active



Workout Plan

Personalized routines



Nutrition Plan

Meal guidance



Progress

Monitor gains



AI Coach

Ask anything



Camera Coach

Live form check

Generated Workouts Plan:

Workout Plan

[← Back](#)

Muscle Gain Program

Level: active | Daily Duration: 75 min | BMI: 24.4

Monday

Chest & Triceps • 52 min

Bench Press

4 sets × 8 reps | Rest: 90s | Intensity: high

[Start Exercise](#)

Incline Dumbbell Press

4 sets × 8 reps | Rest: 90s | Intensity: high

[Start Exercise](#)

Overhead Tricep Extension

4 sets × 8 reps | Rest: 90s | Intensity: high

[Start Exercise](#)

Tricep Pushdown

4 sets × 8 reps | Rest: 90s | Intensity: high

[Start Exercise](#)

Tuesday

Back & Biceps • 52 min

Bicep Curl

4 sets × 8 reps | Rest: 90s | Intensity: high

[Start Exercise](#)

Barbell Row

4 sets × 8 reps | Rest: 90s | Intensity: high

[Start Exercise](#)

Seated Cable Row

4 sets × 8 reps | Rest: 90s | Intensity: high

[Start Exercise](#)

Tricep Pushdown

4 sets × 8 reps | Rest: 90s | Intensity: high

[Start Exercise](#)

Accessing the First Workout:

Tuesday

Back & Biceps • 52 min

Bicep Curl

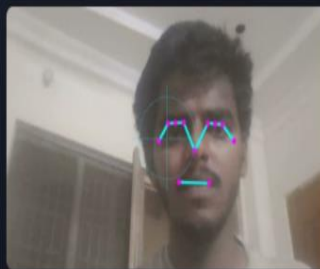
4 sets × 8 reps | Rest: 90s | Intensity: high

Start Exercise

AI Posture Coach

Real-time facial tracking via Intel® OpenPose Engine

Back



Select Exercise

Lunges

Reps

0

Accuracy

100%

Live Feedback

Face camera properly

Session Time

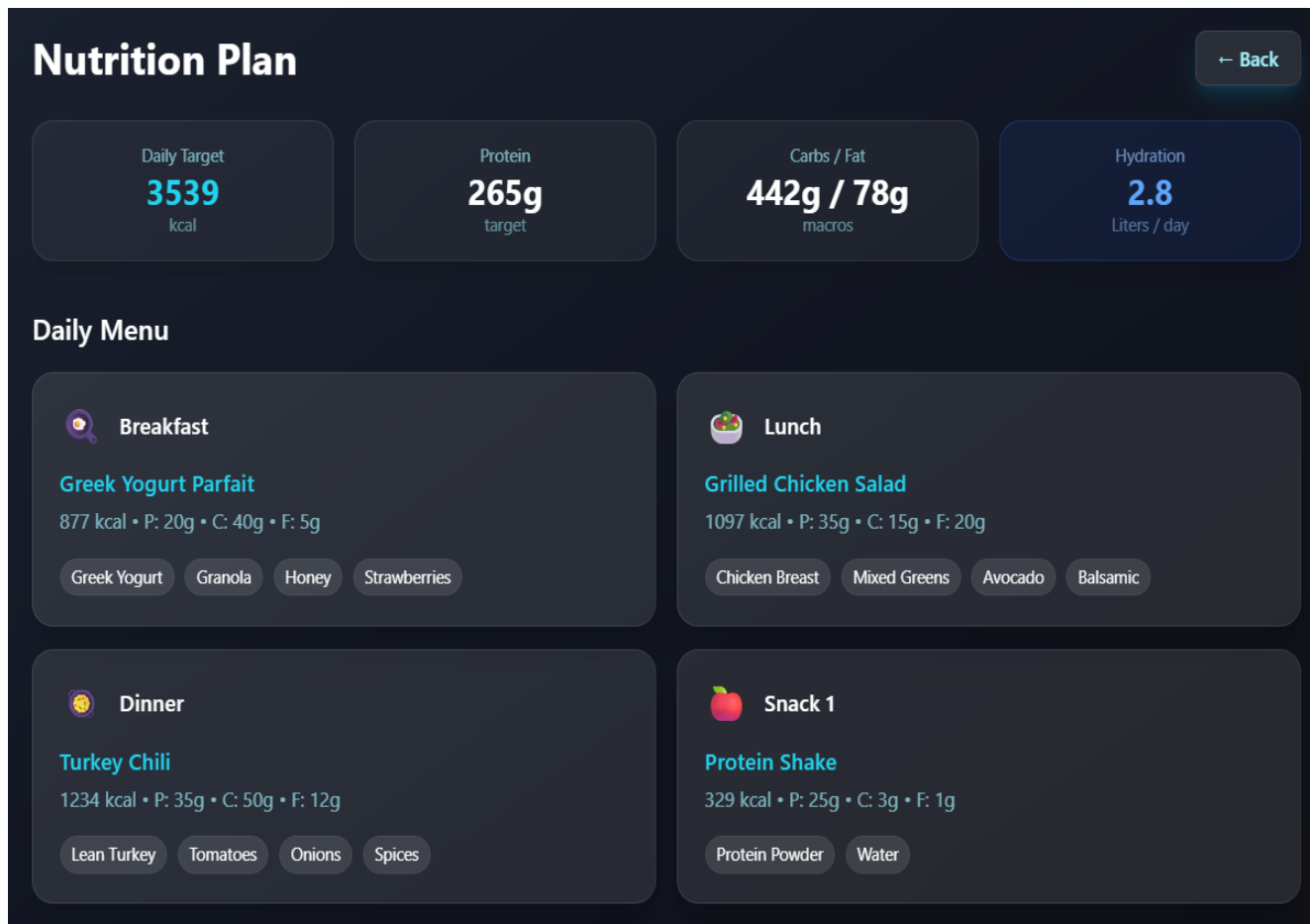
00:41

Engine Status

Local AI Active

End Session & Save Progress

Nutrition Tracker:



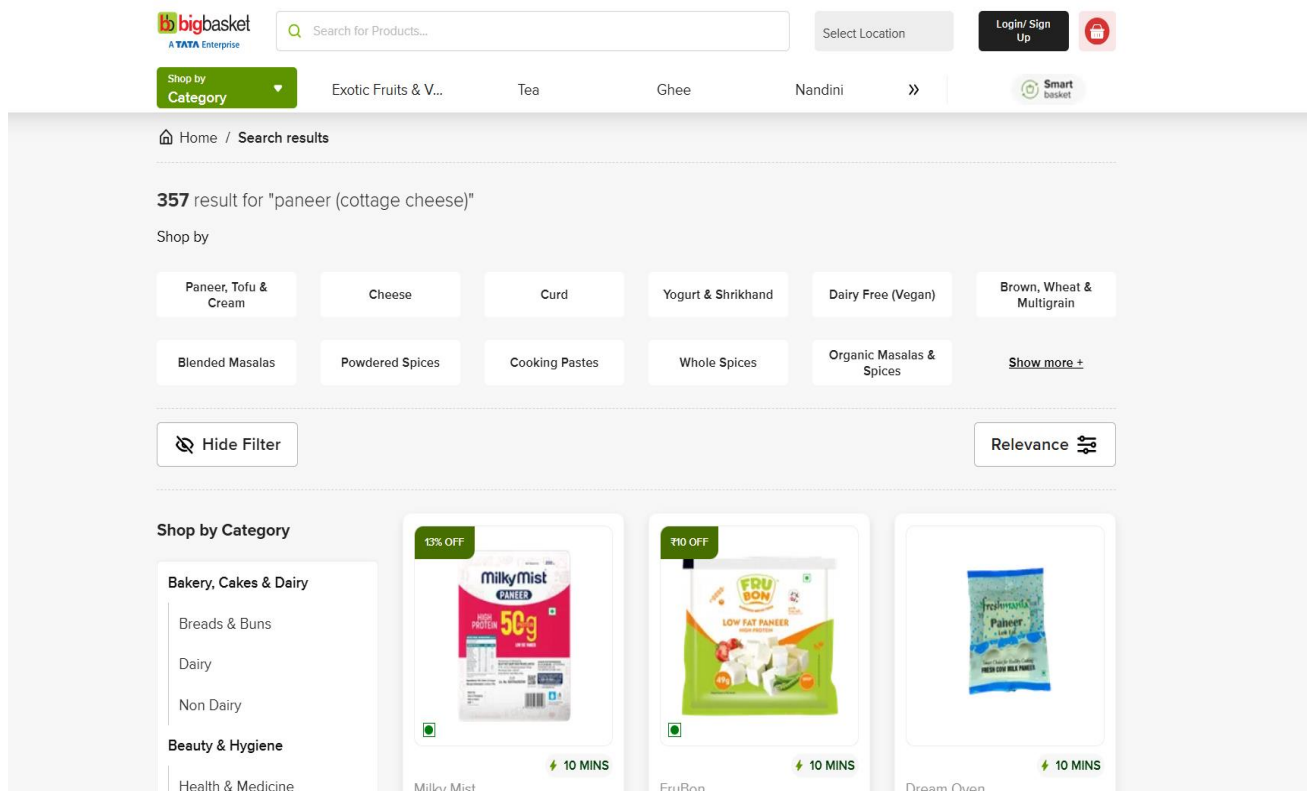
Smart Grocery List:

Smart Grocery List

Powered by BigBasket

Lean Turkey	Shop →	Greek Yogurt	Shop →	Tomatoes	Shop →
Chicken Breast	Shop →	Mixed Greens	Shop →	Granola	Shop →
Strawberries	Shop →	Onions	Shop →	Spices	Shop →
Protein Powder	Shop →	Avocado	Shop →	Honey	Shop →
Water	Shop →	Balsamic	Shop →		

Redirecting to BigBasket for purchasing items:

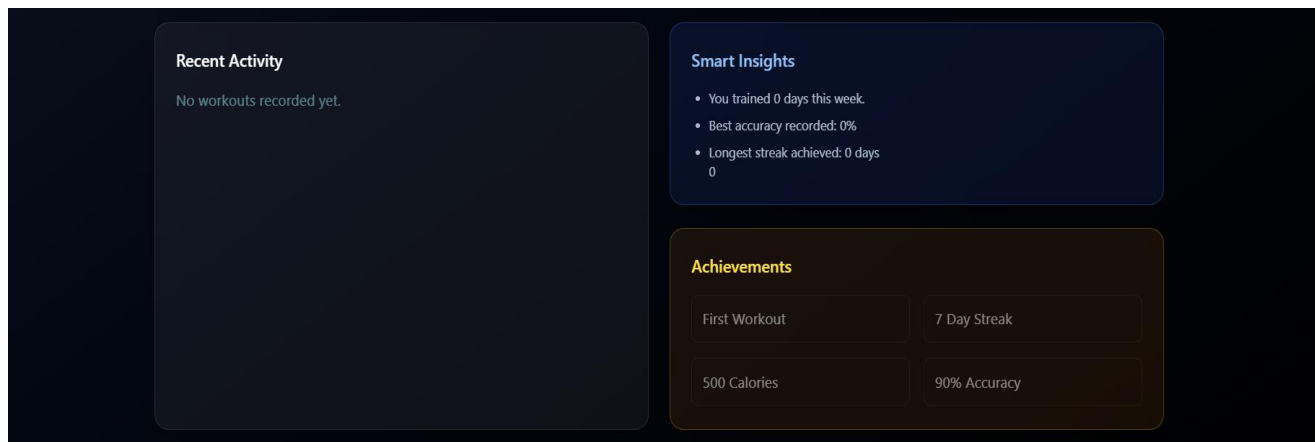


The screenshot shows the BigBasket website interface. At the top, there's a search bar with the text "Search for Products...", a "Select Location" button, and a "Login/Sign Up" button. Below the search bar, there's a "Shop by Category" dropdown menu. The main content area displays "357 result for 'paneer (cottage cheese)'" and a "Shop by" section with various category filters like "Paneer, Tofu & Cream", "Cheese", "Curd", "Yogurt & Shrikhand", "Dairy Free (Vegan)", "Brown, Wheat & Multigrain", "Blended Masalas", "Powdered Spices", "Cooking Pastes", "Whole Spices", and "Organic Masalas & Spices". There's also a "Hide Filter" button and a "Relevance" sorting option. The product listings show three items: "Milky Mist" (50g, 13% OFF, 10 MINS), "FruBon" (Low Fat Paneer, ₹10 OFF, 10 MINS), and "Dream Oven" (Paneer, 10 MINS).

Progress Tracking:



The screenshot shows a "Progress Tracking" dashboard. At the top, there's a title "Progress Tracking" and a subtitle "Monitor your fitness journey and achievements". A "Back" button is in the top right corner. The dashboard features four main metrics: "Current Streak" (0 Days), "Calories Burned" (0), "Total Workouts" (0), and "Total Minutes" (0m). Below these, there are two charts: "Weekly Calories Burned" and "Posture Improvement (Accuracy)". Both charts show a grid for the days of the week (Mon to Sun) but no data is plotted.



Conclusion:

ArogyaMitra successfully brings together AI intelligence, personalized fitness planning, and modern wellness principles into a unified, user-friendly platform. By integrating Groq's advanced LLaMA model, the system generates tailored workout routines, nutrition plans, and real-time adaptive coaching that respond dynamically to each user's lifestyle, preferences, and health conditions. This ensures that every individual receives a fitness experience that is not only effective but also safe, relevant, and deeply personalized.

The platform's modular architecture built with FastAPI, React, SQLAlchemy, YouTube API. Users can explore structured workout plans, maintain healthy eating habits, track their progress, and receive instant assistance from AROMI, the AI wellness coach. The inclusion of a charity-based gamification system adds motivation and a sense of social contribution, enhancing user engagement and consistency.

Through its seamless combination of technology, AI-driven insights, and holistic health features, ArogyaMitra stands as a complete wellness ecosystem designed to support long-term fitness transformation. The project demonstrates how intelligent systems can meaningfully improve personal health journeys by offering adaptive, real-time guidance and empowering users to build sustainable habits. As a scalable and future-ready platform, ArogyaMitra lays the foundation for advanced health innovations, such as wearable integrations, real-time analytics, and AI-assisted coaching at a global scale.

